

Znajdowanie k-tego elementu

(statystyki porządkowe)

We: zbiór S , $|S|=n$, liczba k , $1 \leq k \leq n$

Wy: wartość k -tego co do wielkości elementu zbioru S

W szczególności szukamy elementu

- dla $k=1$ - najmniejszego,
- dla $k=n$ - największego,
- dla n nieparzystej, $k=\lceil n/2 \rceil$ - mediany.

Np. $S=\{8, 12, 3, 7, 15, 1, 18, 9, 4, 17, 14, 11\}$, $k=6$.

wynik: 9

Metoda 1.

- posortować,
- odczytać wartość k -tego elementu.

Łatwa, czasochłonna.

Metoda 2. (zalecana dla k bliskich 1 lub n).

Z użyciem kopca, np. dla k bliskiego 1:

- utworzyć min-kopiec(S),
- wykonać $k-1$ razy deletemin,
- odczytać korzeń – jest to k -ty element,

Złożoność:

tworzenie kopca + $(k-1)$ razy deletemin:

$$\Theta(n + k \lg n)$$

Dla k bliskiego n tworzymy max-kopiec i wykonujemy $n-k$ razy deletemax.

Metoda 3 (Hoare).

Wzorowana na algorytmie quicksort

```
Select (S, k);  
// znajdowanie k-tego co do wielkości elementu zbioru  
// licząc od najmniejszego (k=1 - element najmniejszy)  
  
  a := dowolny element zbioru S;  
  przegladnij zbiór S i podziel na 3 zbiory:  
    S1 := {x∈S : x<a};  
    S2 := {x∈S : x=a};  
    S3 := {x∈S : x>a};  
  if (|S1| ≥ k) return select(S1, k);  
    // szukany element jest k-tym elementem w S1  
  if (|S1| + |S2| ≥ k) return a;  
    // szukany jest w S2, czyli równy a  
  return select(S3, k-|S1|-|S2|);  
    // szukaj w S3, parametr k odpowiednio mniejszy  
end.
```

Przykład:

S = {8, 12, 3, 7, 15, 1, 18, 9, 4, 17, 14, 11}

k = 7, a = 8

S1 = {3, 7, 1, 4}

S2 = {8}

S3 = {12, 15, 18, 9, 17, 14, 11}

rekurencja: S3, k = 2.

Wynik: 11

Złożoność pesymistyczna: $\Theta(n^2)$ (jak dla quicksort-u)

Złożoność średnia ?

Fakt: Oczekiwana liczba porównań $\leq 4n = \Theta(n)$.
Dowód pomijamy

Mniej niż dla quicksortu, bo rekurencyjnie wykonywane tylko jedno podzadanie.

Metoda 4 (mediana median, magiczne piątki)

Modyfikacja metody 3 – lepszy wybór elementu a.

```
SelectMM(S, k);  
// k-ty element, metoda "mediana median"  
  if |S|<50 then  
    posortuj S=S[1..n] niemalejąco;  
    return S[k];  
  podziel S na 5-elementowe podzbiory;  
  posortuj każdy podzbiór oddzielnie;  
  Q := zbiór median wszystkich podzbiorów  
                                     // Q lokalny, |Q| =  $\lceil n/5 \rceil$   
  M := selectMM(Q, n/10)           // mediana median  
                                     // dalej jak w metodzie Hoare'a, a=M  
  przegladnij zbiór S i podziel na 3 podzbiory:  
    S1 := {x∈S : x<M};  
    S2 := {x∈S : x=M};  
    S3 := {x∈S : x>M};  
  if (|S1| ≥ k) return select(S1, k);  
    // szukany element jest k-tym elementem w S1  
  if (|S1| + |S2| ≥ k) return M;  
    // szukany jest w S2, czyli równy M  
  return select(S3, k-|S1|-|S2|);  
    // szukaj w S3, numer k odpowiednio mniejszy  
end.
```

Fakt.

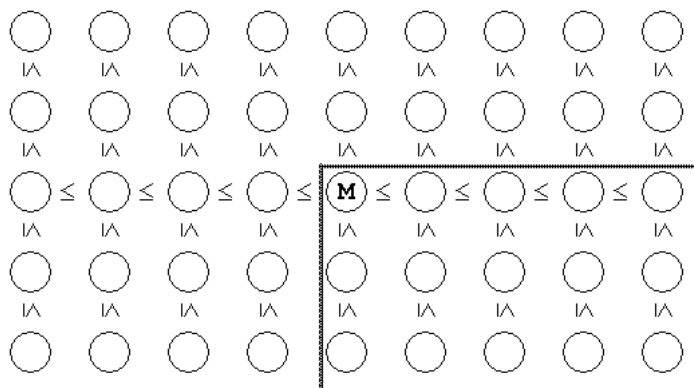
Dla tak wybranego elementu dzielącego na podzadania, moce podzadań spełniają nierówność:

$$|S1| \leq 3/4 * n \text{ oraz } |S3| \leq 3/4 * n.$$

Dowód:

Posortowane 5-elementowe podzbiory wypisujemy obok siebie w kolejności takiej, by:

- wartości w każdym podzbiorze rosły gdy czytamy je z góry na dół (to nasz algorytm oblicza),
- mediany podzbiorów rosły od lewej do prawej (tego algorytm nie oblicza, tak jest tylko na rysunku),
- i wtedy w środku jest mediana median (algorytm ją oblicza).



- elementy prawej dolnej ćwiartki są nie mniejsze niż M ,
- zatem zbiór $S1 = \{ x : x < M \}$ zawiera się w pozostałych trzech ćwiartkach,
- a więc $|S1| \leq 3/4 |S|$.

Analogicznie, $|S3| \leq 3/4 |S|$.

Równanie rekurencyjne opisujące złożoność:

$$T(n) \leq c, \text{ dla } n < 50,$$

$$T(n) \leq T(n/5) + T(3/4 n) + c*n, \text{ dla } n \geq 50,$$

bo:

- jedno podzadanie wielkości $n/5$ (mediana median),
- jedno podzadanie wielkości co najwyżej $3/4 * n$ (S1 lub S3),
- wyodrębnienie podzadań i sortowanie piątek:
koszt liniowy $c*n$, dla pewnej stałej c .

Łatwo indukcją pokazać, że:

$$T(n) \leq 20*c*n = \Theta(n).$$

Przytoczmy twierdzenie MT (o rekurencji uniwersalnej),
wersja liniowa:

Jeśli $c_i > 0, i=1, \dots, m, \sum_{i=1, \dots, m} c_i < 1$,
to rozwiązanie równania

$$T(n) = \sum_{i=1, \dots, m} T(c_i n) + \Theta(n)$$

wynosi $T(n) = \Theta(n)$.

Z tego twierdzenia również wynika, że algorytm
"mediana median" ma złożoność $T(n) = \Theta(n)$.

Złożoność średnia jest tego samego rzędu - oczywiste.

Pytania:

- dlaczego magiczne piątki a nie trójki?
- dlaczego magiczne piątki a nie siódemki (lub dziewiątki itd.)?
- ile porównań potrzeba i wystarcza dla posortowania ciągu długości 5? A długości 7?